

Especificación Técnica: Ecosistema Minecrack (Hub + Launcher)

Este documento define la arquitectura, infraestructura y especificaciones técnicas para la implementación del Hub de Mods y su integración directa con el launcher de Minecrack. El objetivo es evolucionar el proyecto de un cliente local a un ecosistema cliente-servidor robusto.

1. Visión General del Sistema

El sistema se divide en tres componentes principales que interactúan para garantizar la distribución eficiente, segura y versionada de mods y configuraciones a los jugadores:

- **Minecrack Backend (FastAPI):** API RESTful encargada de la lógica de negocio, validación de integridad mediante hashes criptográficos y distribución optimizada de archivos (soporte para descargas por fragmentos).
- **Minecrack Admin Panel (React):** Interfaz web para administradores de la comunidad. Permite la carga de archivos .jar, gestión de versiones de modpacks y publicación de notas de parche (*changelogs*).
- **Minecrack Client (Launcher):** La aplicación cliente (escrita en el stack actual del repositorio). Se comunica con la API para sincronizar el estado local de los archivos con el servidor antes de iniciar el entorno de Java.

2. Stack Tecnológico

Componente	Tecnología Principal	Uso Específico
Backend / API	Python + FastAPI	Manejo asíncrono de rutas, streaming de archivos pesados y validación SHA-256.
Base de Datos	PostgreSQL + SQLAlchemy	Almacenamiento relacional, control de dependencias e historiales de versiones.
Frontend Admin	React + Vite	Dashboard visual e interactivo para

Componente	Tecnología Principal	Uso Específico
		administradores del servidor.
Almacenamiento	S3 Compatible o Volumen Local	Almacén físico para los archivos .jar y .zip.

3. Diseño de Base de Datos (Relacional)

El diseño relacional se enfoca en permitir múltiples versiones de un mismo mod y agrupar estas versiones exactas en un "Modpack" para garantizar que todos los clientes tengan exactamente los mismos archivos.

Entidades Principales

- **Mod:** Contiene metadatos agnósticos a la versión (ID, Nombre, Autor, Enlace original, Descripción).
- **ModVersion:** La instancia física de un archivo (ID, ModID, Versión del Mod, Versión de Minecraft compatible, Hash SHA-256, Tamaño del archivo, URL de Descarga).
- **Modpack:** Una agrupación jugable y versionada (ID, Nombre, Versión del Modpack, Fecha de lanzamiento, Es_Activo).
- **Modpack_ModVersion_Link:** Tabla intermedia que asocia N versiones de mods a M modpacks.

4. Endpoints de la API Central

El launcher se comunicará de forma segura con el servidor mediante los siguientes endpoints REST:

Método	Ruta	Descripción y Propósito
GET	/api/v1/modpacks/active/manifest	Devuelve un JSON con la lista completa de archivos y sus hashes SHA-256 esperados para el modpack actual.

Método	Ruta	Descripción y Propósito
POST	/api/v1/sync/verify	El launcher envía los hashes calculados de la carpeta local; la API devuelve el arreglo de archivos faltantes o corruptos.
GET	/api/v1/download/{file_hash}	Endpoint de streaming que soporta el header Range para descargas concurrentes y pausables.

5. Flujo de Trabajo (Sincronización del Cliente)

El proceso de actualización que ejecutará la aplicación Minecrack al ser abierta por el usuario:

1. El usuario abre **Minecrack Launcher**.
2. El cliente escanea la carpeta local de la instancia (ej. ./minecrack/mods) y genera un hash SHA-256 rápido para cada archivo .jar existente.
3. El cliente hace una petición GET al manifiesto activo del servidor.
4. Compara la lista de hashes locales contra el manifiesto oficial:
 - Identifica archivos obsoletos o no autorizados y los elimina para evitar problemas de compatibilidad.
 - Identifica archivos faltantes o con hashes distintos (corruptos).
5. Inicia la descarga concurrente de los archivos faltantes, mostrando una barra de progreso basada en el tamaño total a descargar.
6. Una vez completada la validación al 100%, se habilita la ejecución del proceso de Java (el botón "Jugar").

6. Esquema SQLAlchemy (Ejemplo de Implementación Backend)

```
from sqlalchemy import Field, SQLAlchemy, Relationship
from typing import Optional, List

class ModVersion(SQLAlchemy, table=True):
```

```
    id: Optional[int] = Field(default=None, primary_key=True)
    mod_id: int = Field(foreign_key="mod.id")
    version_string: str
    minecraft_version: str
    file_name: str
    sha256_hash: str = Field(index=True)  # Indexado para búsquedas
rápidas
    file_size_bytes: int
    download_url: str

class Modpack(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    name: str
    version_string: str
    is_active: bool = Field(default=False)
```

Esta arquitectura asegura escalabilidad y elimina por completo el "infierno de dependencias" en la comunidad, garantizando que el servidor y los clientes ejecuten versiones idénticas.